

Lab & Activities Week 07 Google ADK

Integrantes:

José Martínez-02220131016

Institución: Universidad de Santander

Pregrado: Ingeniería Se Sistemas

Docente: Jorge Galvis

03 de Octubre 2026

Activity 1

Activity 1 · Warm-up Concept Quiz10 MININDIVIDUALFORMATIVE

1. Which ADK object actually executes the model-call / tool-call loop for one user turn?

- ☒ SessionService
- ☐ Runner
- ☐ ToolContext

Correct answer: B. The Runner drives the loop and streams events; SessionService only stores state.

2. A tool needs to remember a user's preference across completely separate future conversations. Which state key prefix is correct?

- ☐ temp:
- ☐ app:
- ☐ user:

Correct answer: C. user: state is tied to user_id and persists across that user's sessions, not shared with other users.

3. Why must a tool function's parameters be type-hinted?

- ☐ ADK derives the tool's JSON schema (what the model sees) from the type hints
- ☐ Python requires type hints for all functions
- ☐ It only affects IDE autocomplete, not the agent's behavior

Correct answer: A. Without hints, ADK cannot build a reliable schema, and the model won't know the expected argument types.

4. Which file is **not** optional in an adk-web-discoverable agent folder?

- ☐ README.md
- ☐ __init__.py
- ☐ tests.py

Correct answer: B. __init__.py (plus agent.py with a root_agent) is what adk web scans for.

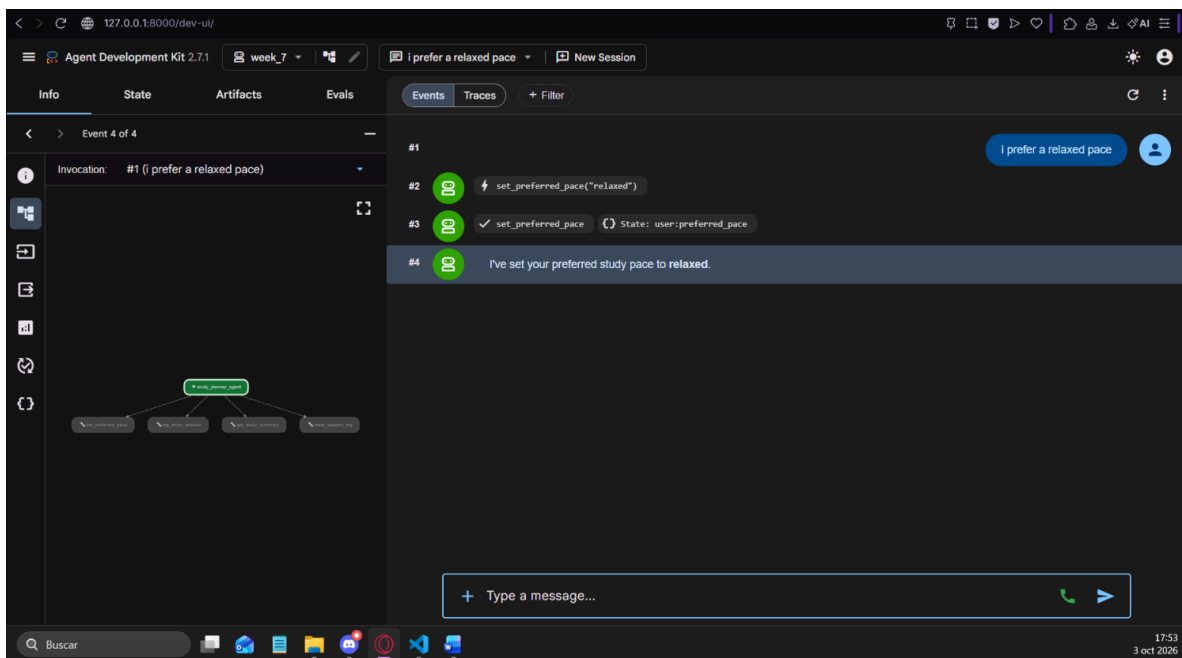
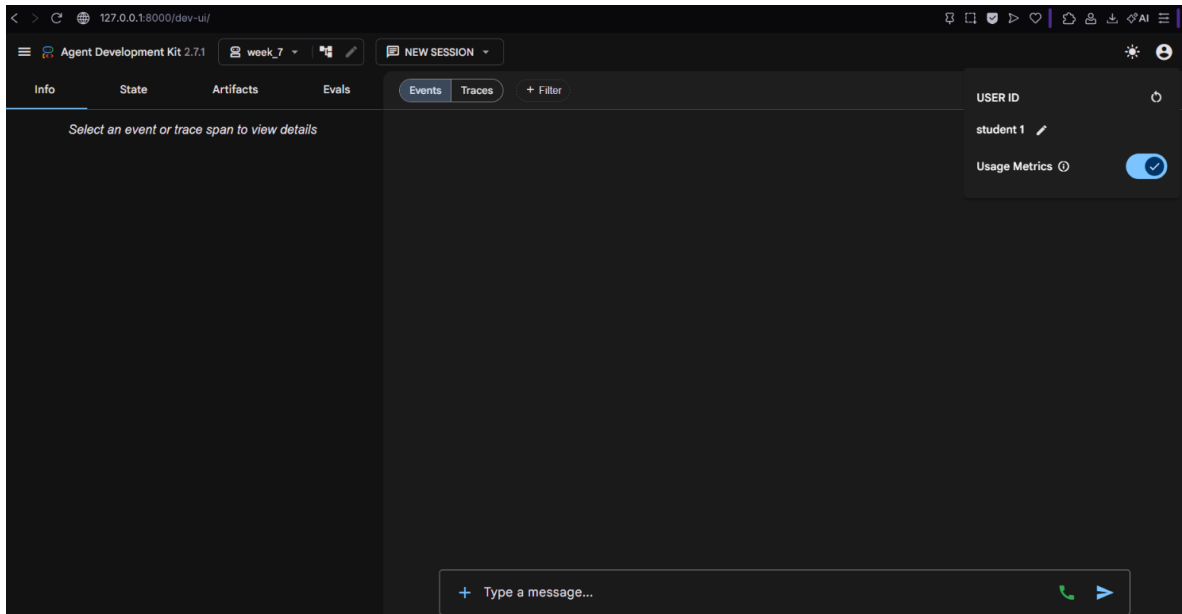
Check answersScore: 4 / 4

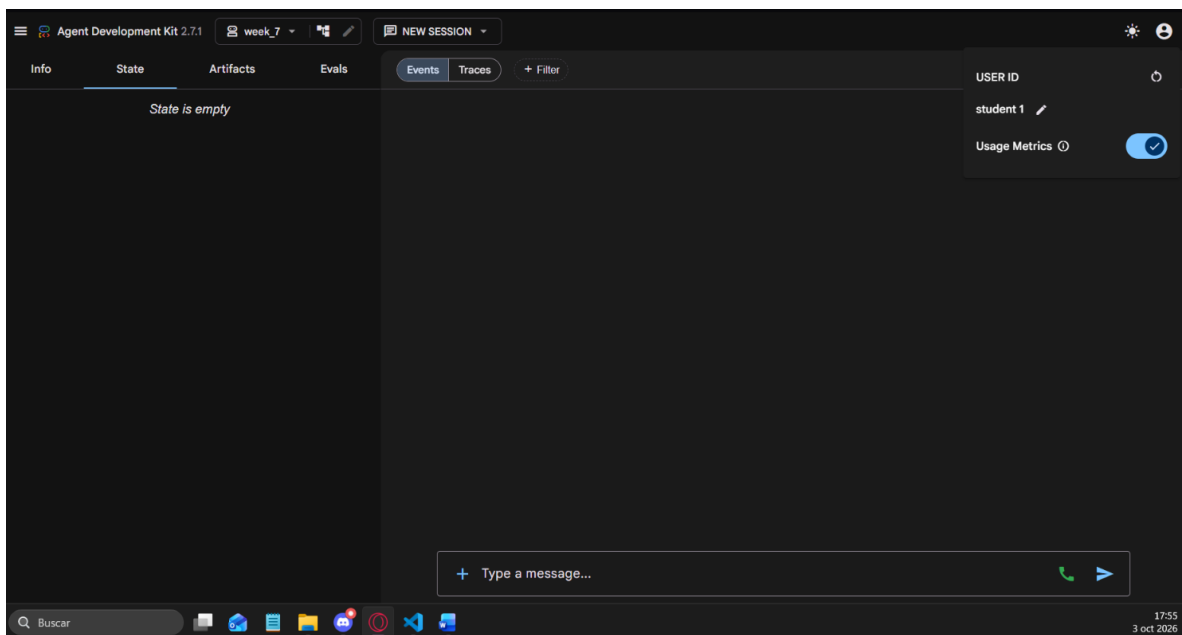
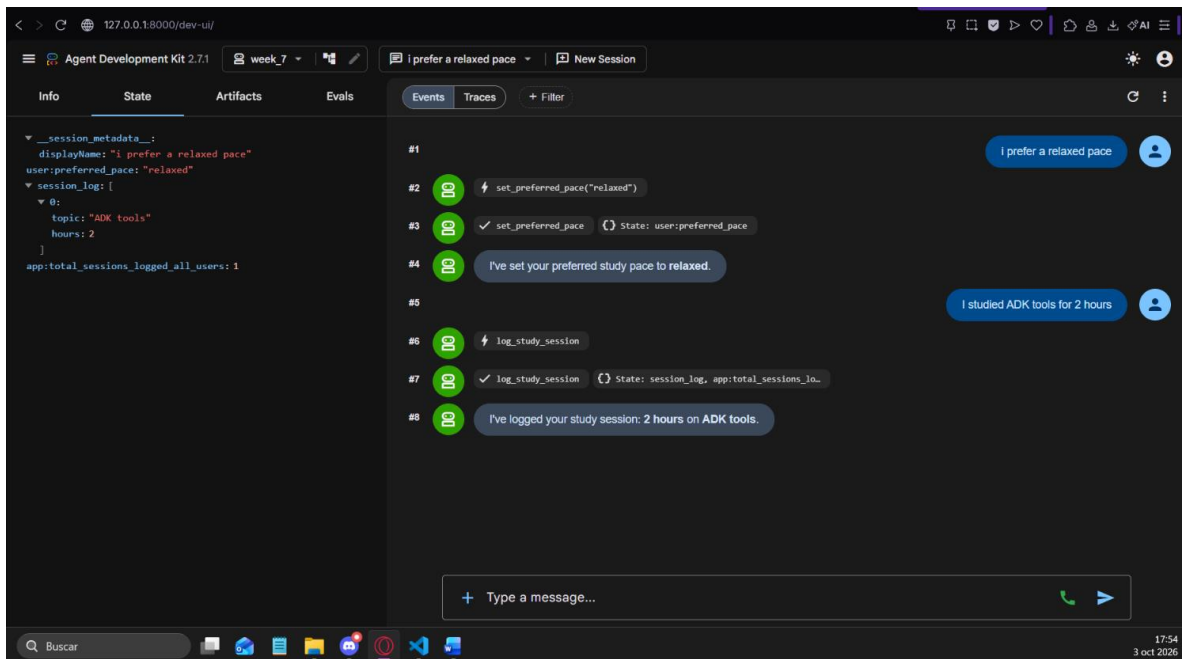
Activity 2

Built agents/week-07-google-adk/adk/study_planner_agent/ live, from an empty folder, following Section 4 of the content page: added set_preferred_pace, log_study_session,

and `get_study_summary` one tool at a time, testing each in adk web before adding the next.

Confirmed state scoping end to end: after setting a pace and logging a session, started a brand-new session with the same `user_id`. `user:preferred_pace` carried over into the new session exactly as expected, while `session_log` came back empty -- matching each key's real scope lifetime.





Discussed why langchain/app.py's "memory" is three plain Python dicts (user_state, app_state, session_state) instead of real scoped session state: LangChain itself has no built-in equivalent to ADK's ToolContext.state. That is an honest simplification for this stage of the course, not a framework limitation -- the real analog is LangGraph's checkpointer, covered in Week 9.

Activity 3

No group was available for this session, so the four scope-assignment cases below were worked through individually.

- 1. The user's preferred UI language -> user:** scope. It is a personal preference tied to that user_id and must follow the user into every future session, not reset each conversation or leak to other users.
- 2. A raw API response a tool needs to pass to another tool call two steps later, within the same turn -> temp:** scope. It only needs to exist for the rest of this one turn's tool-call chain; it should not survive into the next turn, let alone the next session.
- 3. The number of currently active free-tier API quota units remaining today, shared by all users -> app:** scope. It is a single counter owned by the application as a whole, not by any individual user or session.
4. Which documents the user uploaded in this conversation for RAG (Week 4 recap) -> no prefix (session scope). It must survive every turn of the current conversation, which rules out temp:, but it should not follow the user into a brand-new session the way user: would.

The most debatable case is #4: temp: looks tempting because the data is conversation-scoped and not permanent, but temp: is explicitly turn-only in ADK, so a no-prefix session key is the correct match for something that must outlive a single turn but not the whole conversation history.

Activity 4

Added a fourth tool, `reset_session_log(tool_context)`, to `agents/week-07-google-adk/adk/study_planner_agent/agent.py`. It clears only `tool_context.state["session_log"]` for the current session, leaving `user:preferred_pace` and the `app:total_sessions_logged_all_users` counter completely untouched.

The app:-scoped counter, `app:total_sessions_logged_all_users`, was already being incremented inside `log_study_session` from the live-coding build (Activity 2); `get_study_summary` reports it alongside the per-conversation totals.

Reproduced `reset_session_log` in `langchain/app.py`'s simplified `session_state` dict, with an inline code comment noting it stands in for ADK's session-scoped `tool_context.state["session_log"]` reset; the existing `app_state` increment inside `log_study_session` is likewise commented as the stand-in for ADK's app: scope.

127.0.0.1:8000/dev-ui/

Agent Development Kit 2.7.1 week_7 I prefer an intense pace New Session

Info State Artifacts Evals

Events Traces + Filter

#4 I have updated your preferred study pace to intense.

#5 I studied Google ADK for 3 hours

#6 log_study_session

#7 log_study_session State: session_log, app:total_sessions_lo...

#8 I have logged your study session: 3 hours on Google ADK.

#9 give me my study summary

#10 get_study_summary()

#11 get_study_summary

#12 Here is your study summary:

- Preferred Pace: intense
- Sessions This Conversation: 1
- Total Hours This Conversation: 3 hours
- Total Sessions Logged (All Users): 2

+ Type a message...

Buscar

17:57
3 oct 2026

127.0.0.1:8000/dev-ui/

Agent Development Kit 2.7.1 week_7 I prefer an intense pace New Session

Info State Artifacts Evals

Events Traces + Filter

#8 I have logged your study session: 3 hours on Google ADK.

#9 give me my study summary

#10 get_study_summary()

#11 get_study_summary

#12 Here is your study summary:

- Preferred Pace: intense
- Sessions This Conversation: 1
- Total Hours This Conversation: 3 hours
- Total Sessions Logged (All Users): 2

#13 reset my session log please

#14 reset_session_log()

#15 reset_session_log State: session_log

#16 Your study log for this conversation has been cleared. Your preferred pace remains saved.

+ Type a message...

Buscar

17:58
3 oct 2026